Title: Message Intelligence System
Duration: 6 Hours

🎯 Objective

The objective of this project is to design a **classification system** that identifies whether incoming digital messages are **Spam or Legitimate**. Students will combine **probability theory** with **distance-based, margin-based, and probabilistic classifiers**, and analyze how different assumptions impact model performance.

📑 Problem Statement

You are working as a **Data Scientist** for a communication security company. The company wants to build a **Message Intelligence System** that can automatically classify user messages as:

- **0 → Legitimate Message**
- **1 → Spam Message**

The dataset contains **message-related features** extracted from text (numerical summaries) and user behavior signals. Some features are highly correlated, while others follow probabilistic patterns. Your task is to **build and compare multiple classification models** and explain their performance using **probability concepts**.

🧠 Part A: Probability & Conceptual Foundation (Theory)

Answer briefly (2–4 lines each):

1. What is **Conditional Probability**?
2. Explain **Bayes' Theorem** and its importance in classification problems.

3. What assumptions does the **Naive Bayes Classifier** make?
4. Explain the working principle of:
    - **K-Nearest Neighbors (KNN)**
    - **Support Vector Machine (SVM)**
5. Compare **distance-based**, **probabilistic**, and **margin-based** classifiers.

🧪 Part B: Dataset Understanding & Preparation

You are provided with a dataset containing:

- Message length
- Number of special characters
- Number of URLs
- Keyword frequency score
- Sender activity score
- Time-based features
- Target variable: **Spam_Label**

Tasks:

6. Identify input features and target variable.
7. Perform basic data preprocessing (scaling where required).
8. Split the dataset into **training and testing sets**.

📊 Part C: Baseline Model – K-Nearest Neighbors

9. Implement **K-Nearest Neighbors (KNN)** classifier.
10. Experiment with different values of **K**.
11. Analyze how distance metrics affect predictions.
12. Identify cases where KNN misclassifies messages.

⚙️ Part D: Support Vector Machine Classifier

13. Implement **Support Vector Machine (SVM)** classifier with:
    - Linear kernel

- RBF or Polynomial kernel

14. Analyze margin separation and support vectors.
15. Compare SVM performance with KNN.

🧱 Part E: Naive Bayes Classifier & Probability

16. Implement **Naive Bayes Classifier**.
17. Manually compute **conditional probabilities** for a few sample messages.
18. Demonstrate how **Bayes' Theorem** is applied to compute class probabilities.
19. Compare theoretical probability calculations with model predictions.

📈 Part F: Model Comparison & Evaluation

20. Evaluate all models using appropriate classification metrics:
    - Accuracy
    - Precision
    - Recall
    - F1 Score
21. Compare:
    - KNN vs SVM vs Naive Bayes
22. Identify which model performs best for:
    - High precision
    - High recall

🧾 Part G: Final Analysis & Reporting

23. Prepare a final report summarizing:
    - Strengths and weaknesses of each classifier
    - Impact of probability assumptions in Naive Bayes
    - Trade-offs between interpretability and performance
    - Business recommendation for real-world deployment
24. Submit:
    - Source code / Jupyter Notebook
    - Evaluation metrics and plots

- Working Streamlit app
- Final conclusions

✅ Submission Guidelines

- Include practical implementation in a Jupyter Notebook and screenshots.
- Label all charts clearly and write short interpretations under each result.
- **GitHub Repository:**
  - Create a GitHub repository to host your project.
  - Upload your project files, including source code and documentation, to the repository along with a live Streamlit app URL.
  - Add a document (PDF) explaining theory concepts with definitions (if any).
  - Ensure that you provide a clear and descriptive README.md file.

Marking Criteria (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb + Streamlit App)	5 Marks	50%	Completed notebook — all tasks, Restart & Run All clean Live working Streamlit App URL
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + screenshots + README + video link
TOTAL	**10 Marks**	**100%**	

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook screen simultaneously. Explain concepts verbally throughout — do not just execute cells without speaking.

- **You can use OBS Studio software for recording.**
- **Submit your video link only via a Google Drive sharable link or via a YouTube unlisted link.**

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!

Message Intelligence System

Supervised Learning

BRING ON YOUR CODING ATTITUDE